Pulling Walmart Product Data with SearchApi.io

Into Azure Data Factory via a REST Linked Service, and Landing the Results in Blob Storage

---

This document walks through the full process captured in the screenshots: finding the Walmart Search endpoint on SearchApi.io, grabbing an API key, wiring that endpoint into Azure Data Factory as a REST linked service and dataset, running a Copy Data pipeline to land the JSON response into Azure Blob Storage, and finally inspecting the resulting JSON structure.

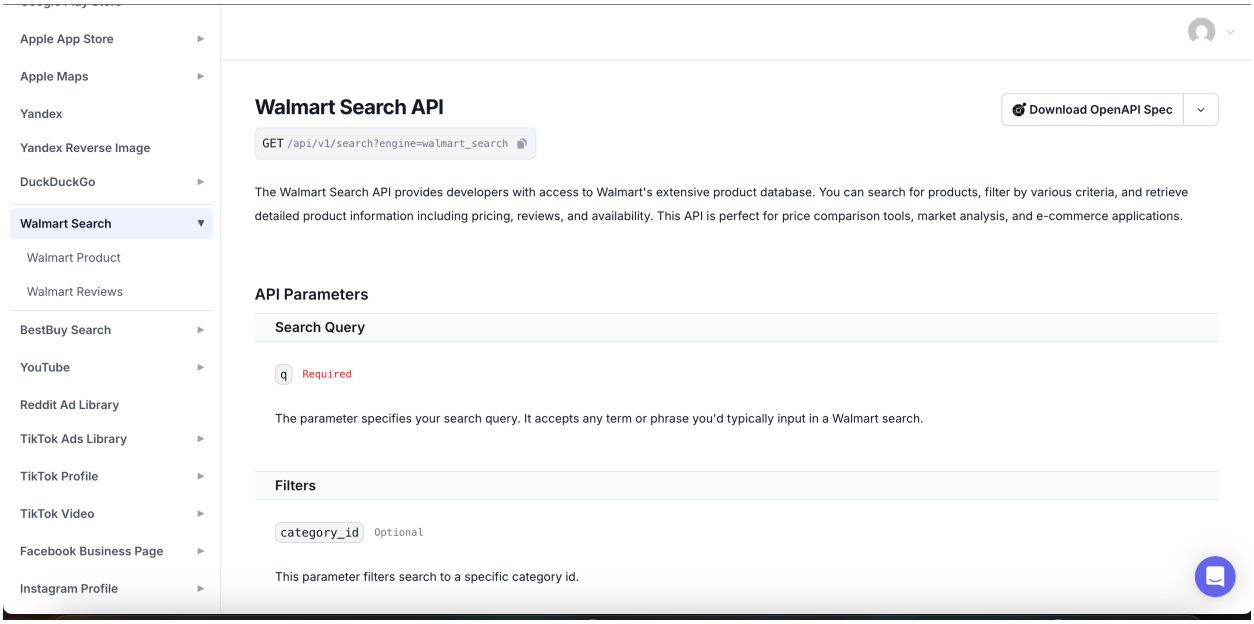
Tools involved: SearchApi.io | Azure Data Factory | Azure Blob Storage | a JSON viewer (jsonviewer.stack.hu)

Step 1: Find the Walmart Search endpoint on SearchApi.io

SearchApi.io is a service that wraps various search engines and marketplaces (Google, Walmart, Best Buy, YouTube, etc.) behind a single, simple REST API. In the left-hand navigation, the **Walmart Search** engine is selected, which reveals the endpoint documentation:

`GET /api/v1/search?engine=walmart_search`

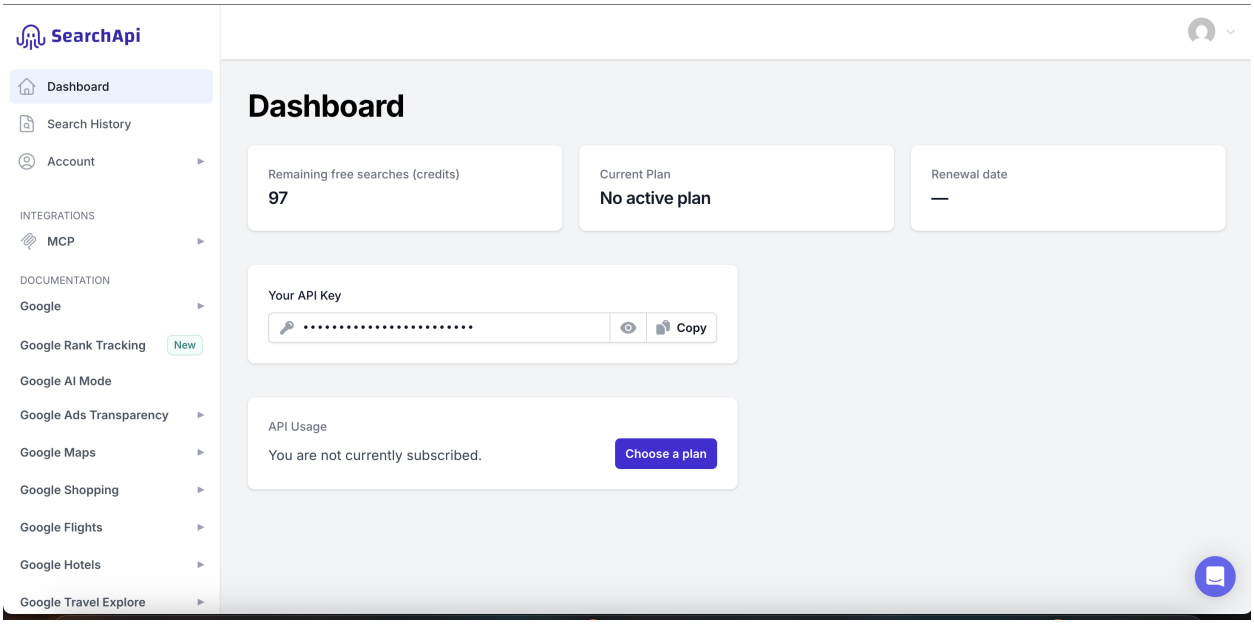
The documentation lists the accepted parameters — the only required one is **q**, the search query (e.g. "electronics"). Optional parameters like **category_id** let you narrow the search further. This page is essentially the contract for what URL to build later.



SearchApi.io documentation page for the Walmart Search API.

Step 2: Grab an API key from the SearchApi dashboard

Every request to SearchApi needs an **api_key** query parameter to authenticate. That key lives on the account Dashboard, alongside the remaining free-search credit balance (97 in this case, since no paid plan is active yet). The key can be revealed and copied directly from this screen.



SearchApi Dashboard: API key, remaining credits, and current plan.

Treat this key like a password — anyone with it can spend your search credits. Avoid pasting it into public documents, shared chats, or client-side code.

Step 3: Assemble the full request URL

With the endpoint shape from Step 1 and the key from Step 2, the pieces are combined into one complete request URL. The example used here searches for "electronics", sorted by best seller:

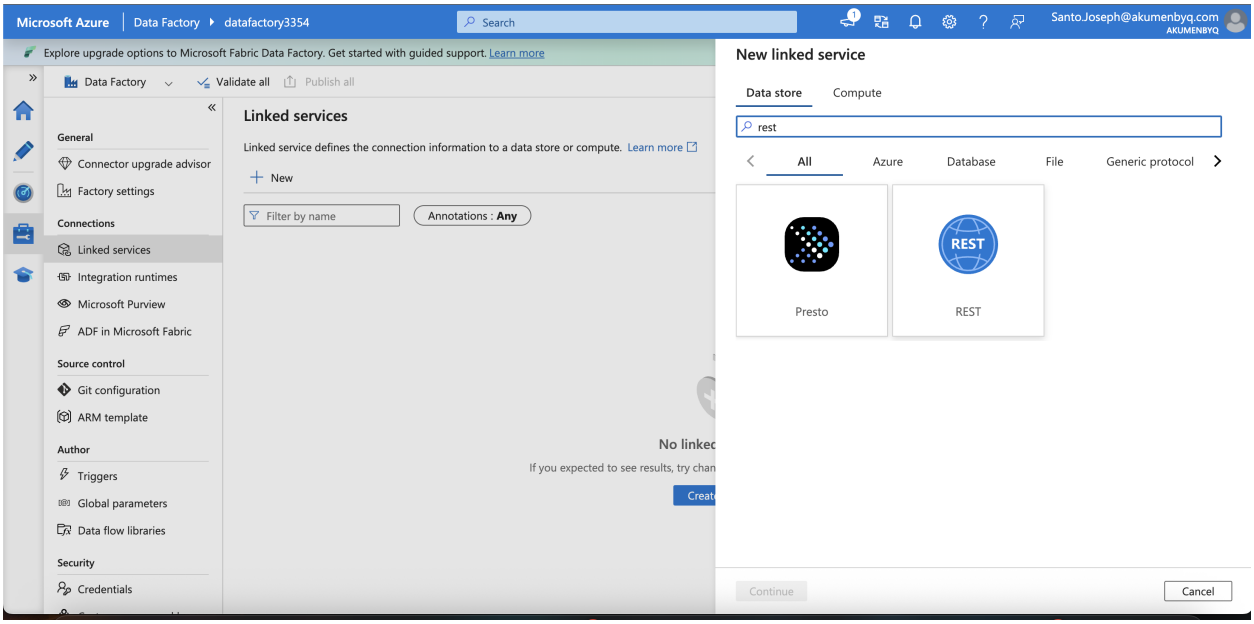
```
https://www.searchapi.io/api/v1/search?engine=walmart_search&q=electronics&sort_by=best_seller&api_key=[REDACTED]
```

(The API key value has been redacted here for safety — in the original notes it followed **api_key=** as a single alphanumeric string.)

This is the exact URL that Azure Data Factory will be configured to call in the next steps. Note the query string structure: **engine** selects the Walmart search engine, **q** is the search term, **sort_by** orders the results, and **api_key** authenticates the call.

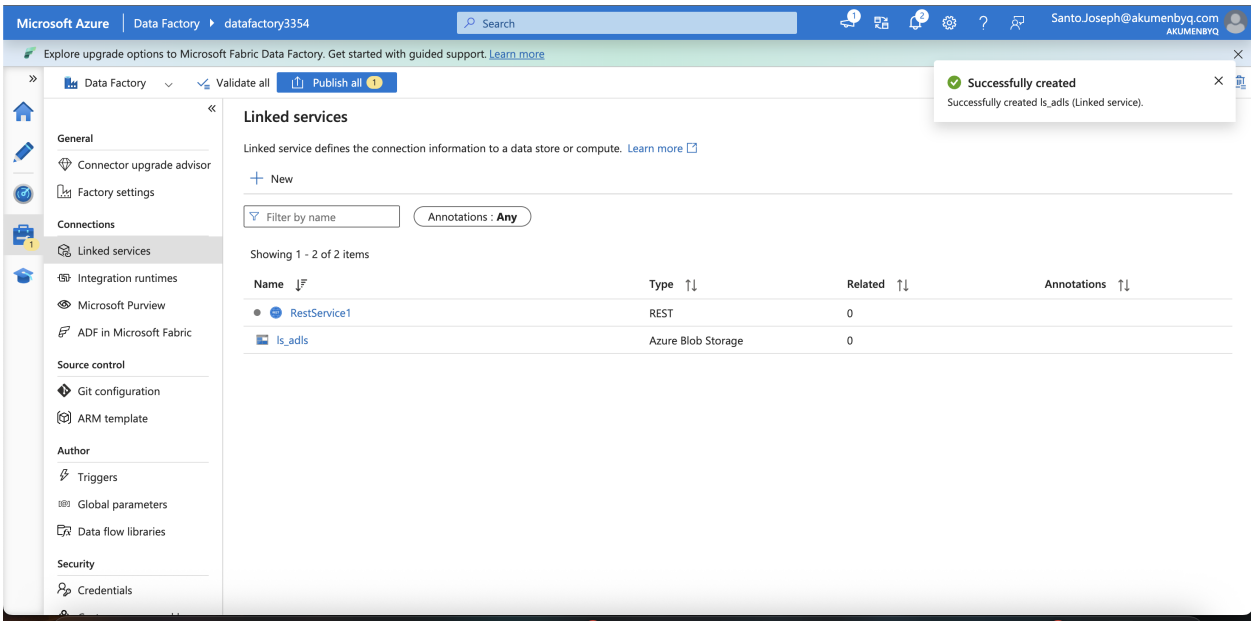
Step 4: Create a REST linked service in Azure Data Factory

In Azure Data Factory (Author → Manage → Linked services → New), searching "rest" in the connector gallery surfaces the generic **REST** data store connector. This is the connector used to talk to any external REST API, including SearchApi.



Choosing the REST connector when creating a new linked service.

A linked service named **RestService1** is created here, pointed at the base URL **https://www.searchapi.io/api/v1**. A second linked service, **ls_adls** (Azure Blob Storage), is also created at this stage — this will later act as the destination for the copied data.

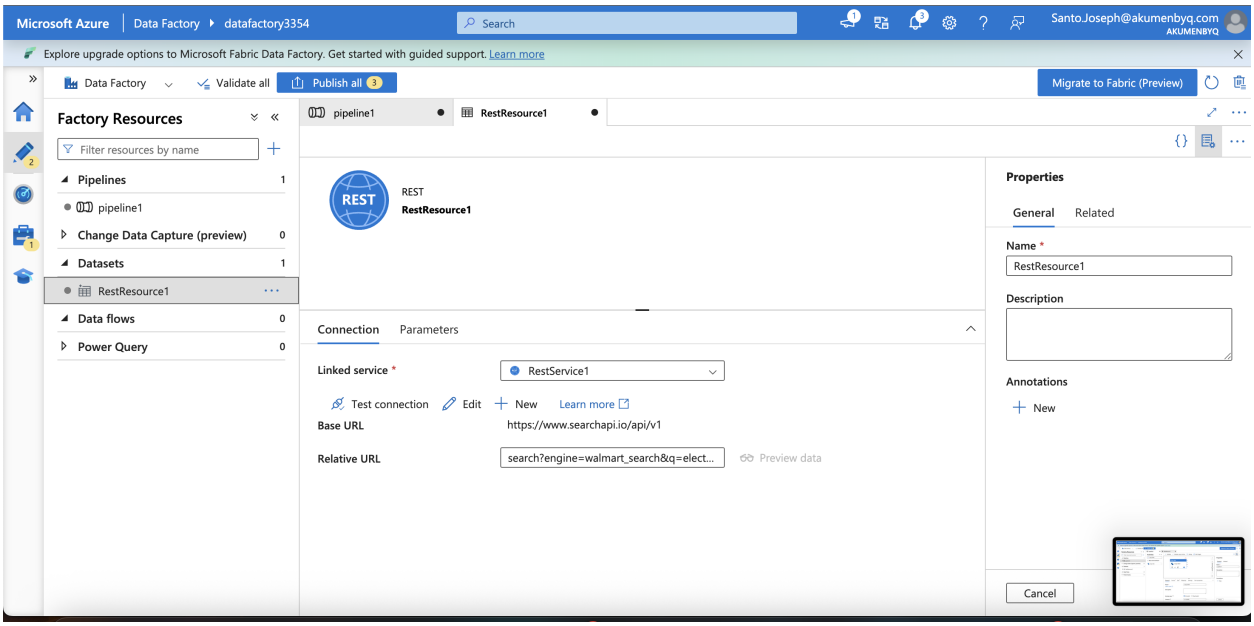


Both linked services now exist: RestService1 (REST) and ls_adls (Azure Blob Storage).

Step 5: Create the source dataset (RestResource1)

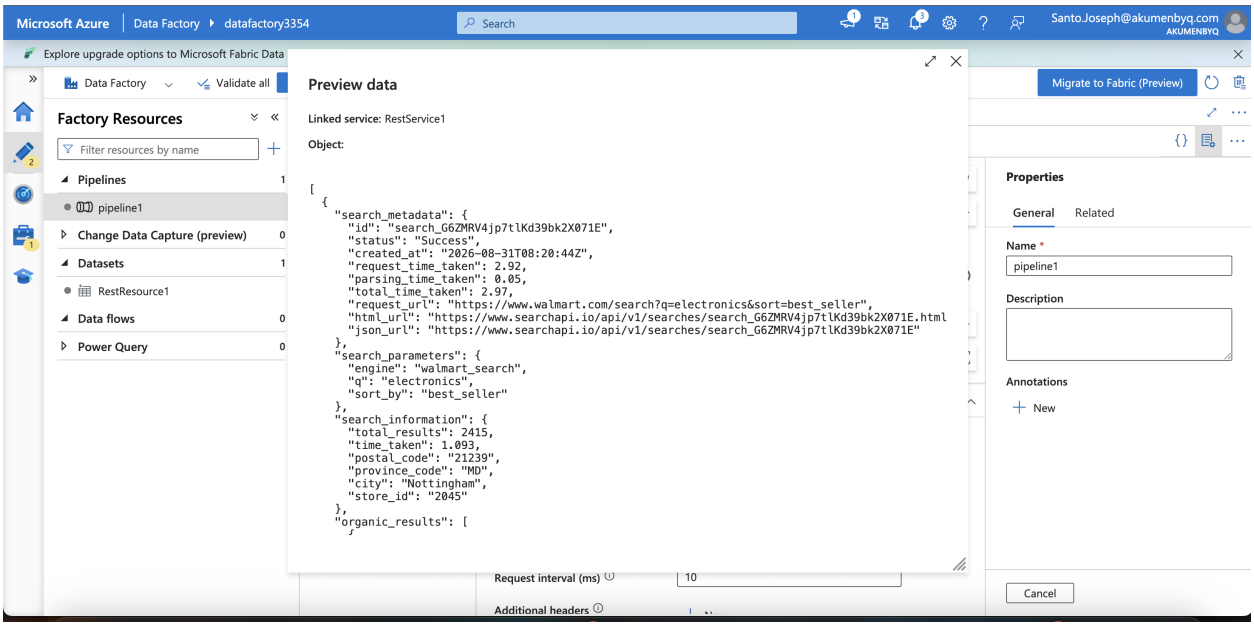
A dataset called **RestResource1** is created on top of the RestService1 linked service. This is where the rest of the URL — everything after the base URL — is configured as the **Relative URL**:

```
search?engine=walmart_search&q=electronics&sort_by=best_seller&api_key=[REDACTED]
```



RestResource1 dataset: Base URL from the linked service, Relative URL holds the query string.

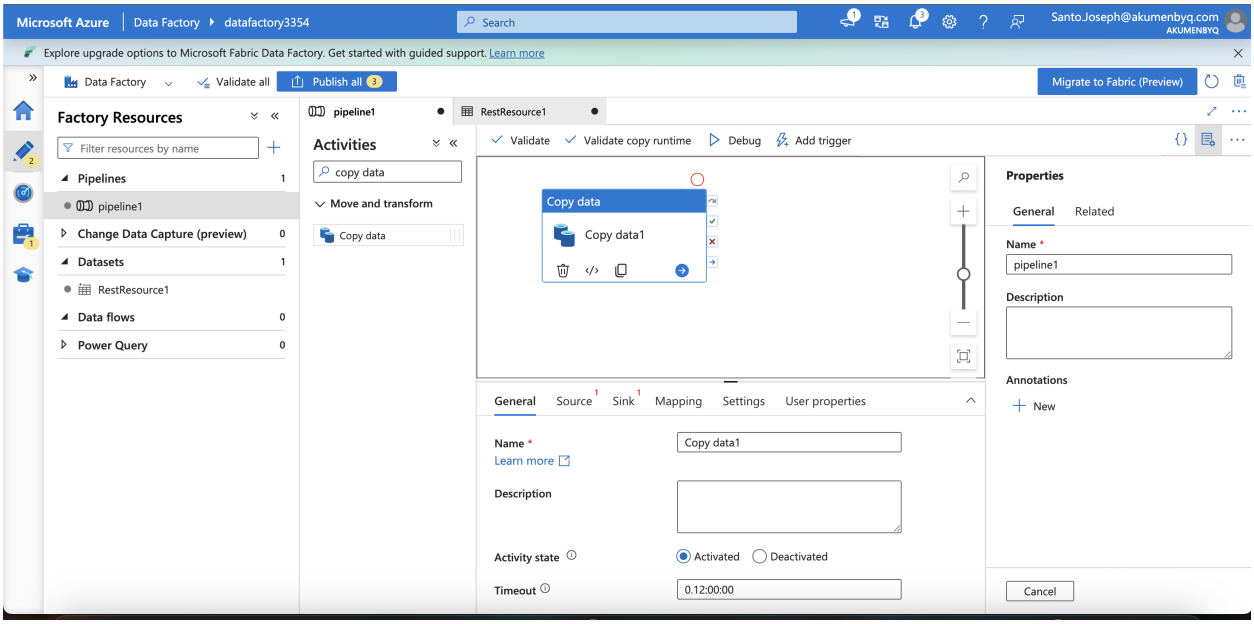
Clicking **Preview data** sends a live test request through this configuration and shows the raw JSON that Walmart / SearchApi returns — useful for confirming the connection works before building the rest of the pipeline.



Live preview of the JSON response, including search_metadata, search_parameters, and search_information.

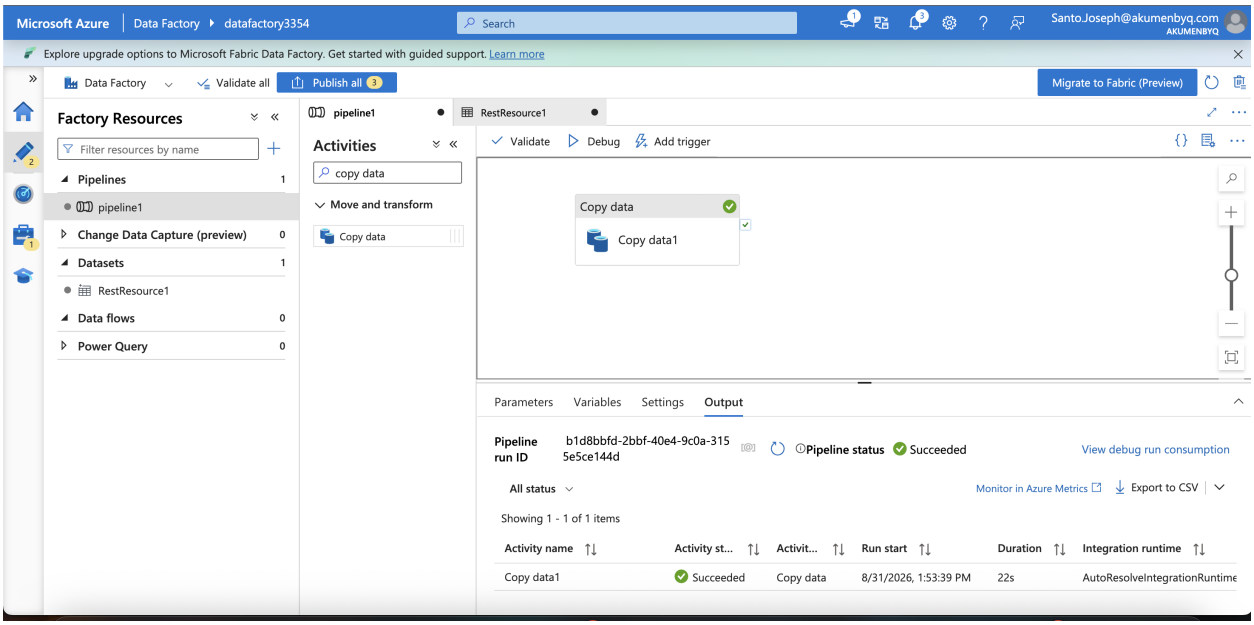
Step 6: Build the pipeline: a Copy Data activity

A pipeline named **pipeline1** is created with a single **Copy data** activity ("Copy data1"). The activity's **Source** tab is pointed at the RestResource1 dataset built in Step 5.



pipeline1 with a Copy data activity in the canvas.

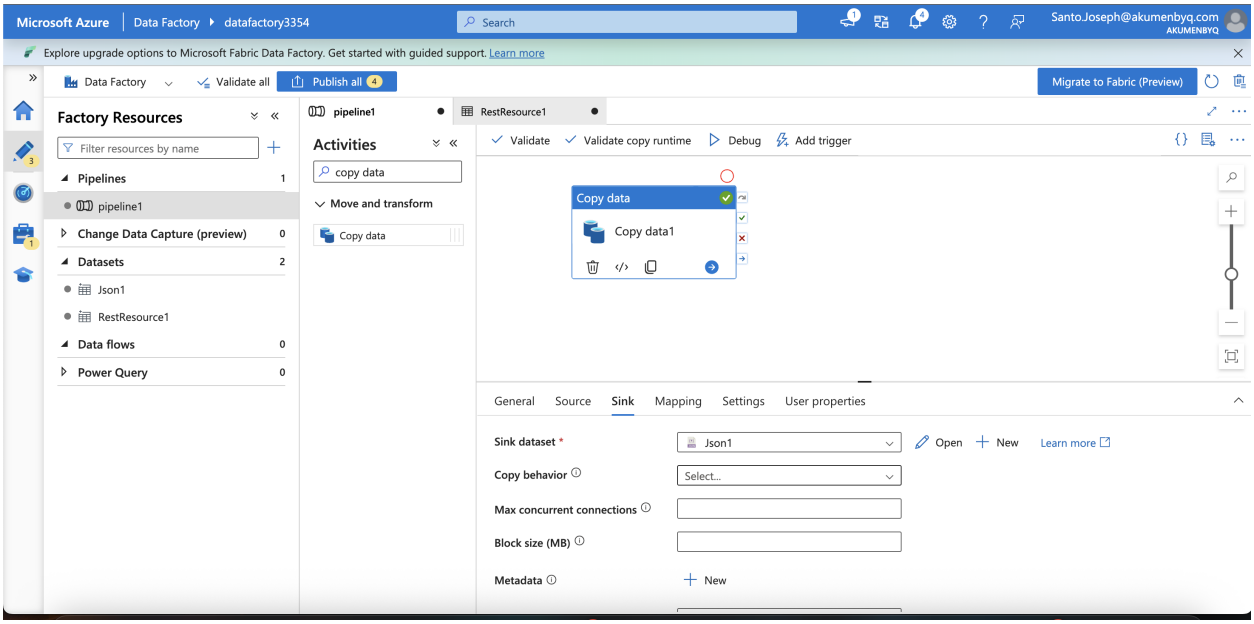
Running **Debug** at this point (with only a source configured) confirms the activity can reach the API successfully — the pipeline run status comes back **Succeeded**.



First debug run: pipeline succeeded after connecting the REST source.

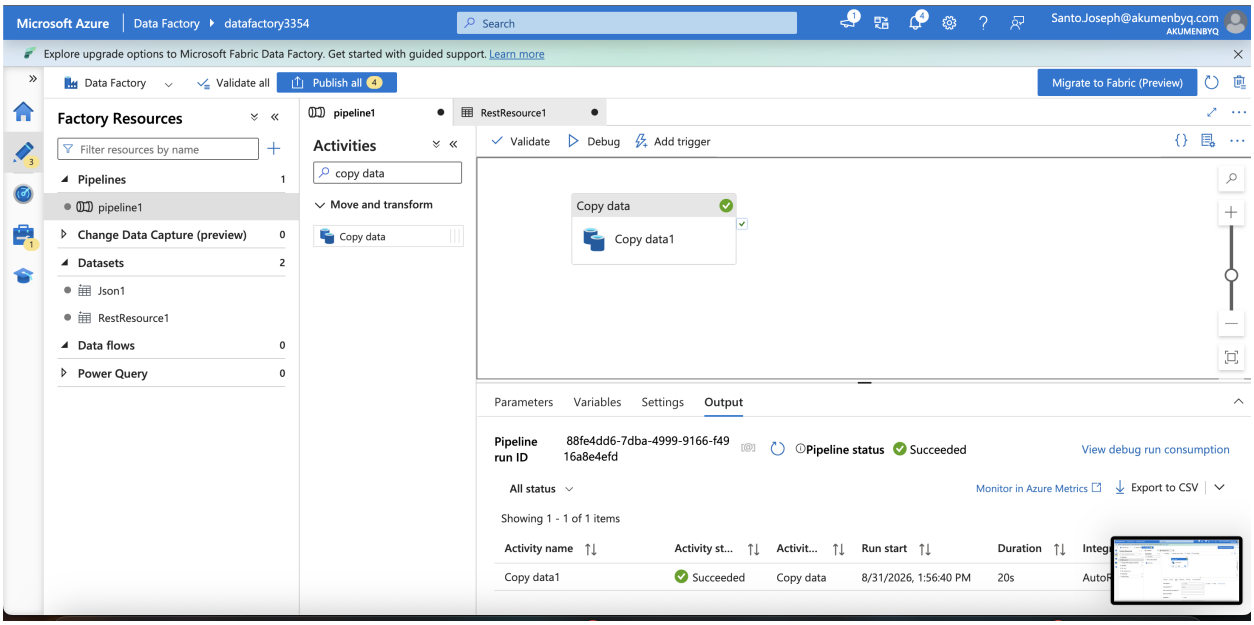
Step 7: Add a Sink: land the JSON in Blob Storage

A second dataset, **Json1**, is created on top of the **ls_adls** (Azure Blob Storage) linked service. This becomes the **Sink** for the Copy data activity, meaning the JSON pulled from the REST source on each run is written out to a blob container.



Sink tab of the Copy data activity, configured to write to the Json1 dataset.

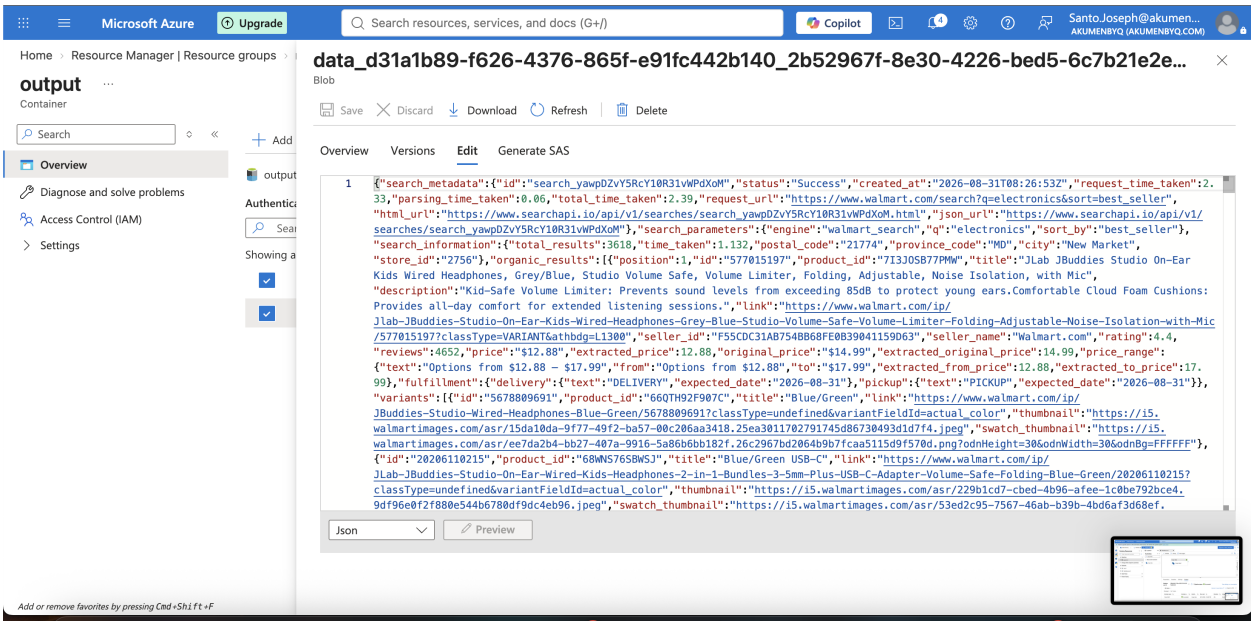
Running **Debug** again with both source and sink wired up succeeds, and the Factory Resources panel now shows 2 datasets (RestResource1 and Json1) instead of 1.



Second debug run: source and sink both configured, pipeline succeeded again.

Step 8: Verify the output landed in Blob Storage

Switching over to the Azure Storage account, the **output** container now holds a blob (named with a generated GUID) containing the raw JSON returned by the API call. Opening it in the portal's built-in Edit view confirms the full Walmart search response — product titles, prices, ratings, links, images, and variant details — was copied over intact.

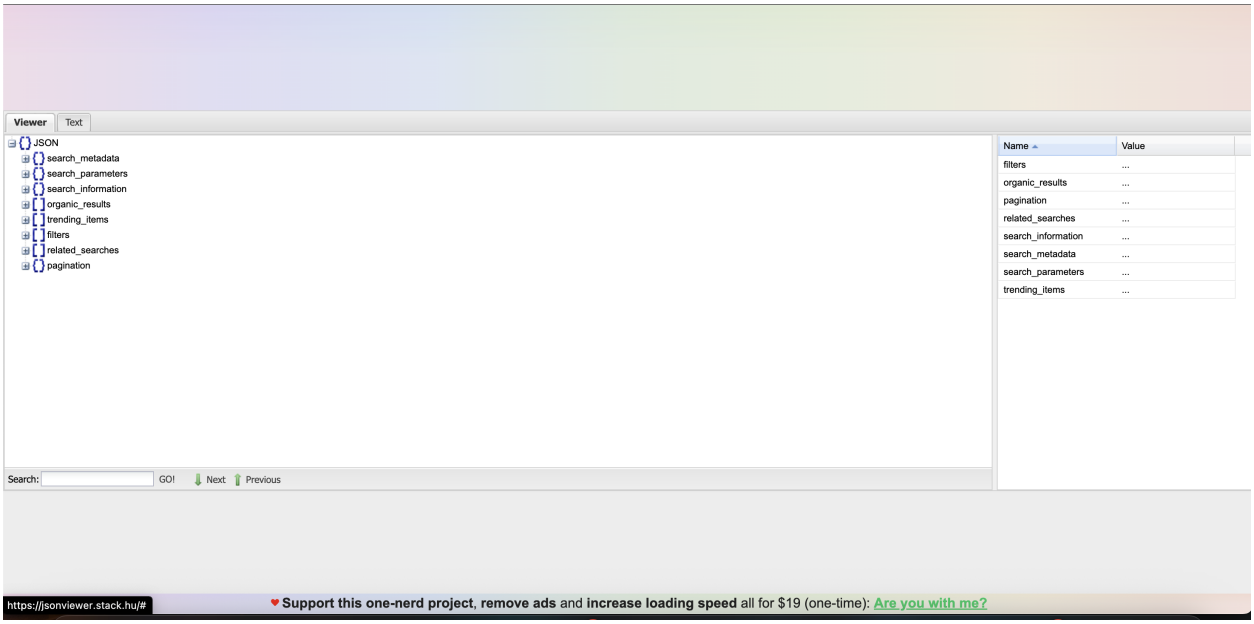


The copied JSON blob viewed and edited directly in the Azure Portal.

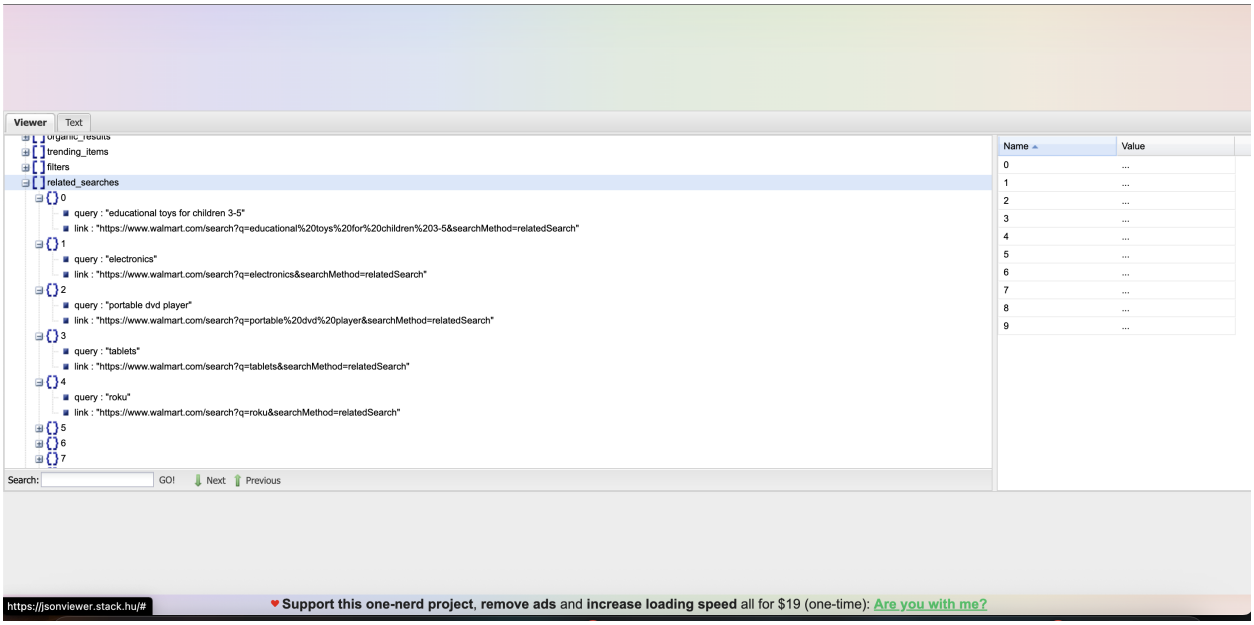
Step 9: Inspect the JSON structure

To make sense of the response shape, the JSON is dropped into a JSON viewer (jsonviewer.stack.hu), which renders it as a collapsible tree. The top level of a Walmart Search API response breaks down into these keys:

Key	Contents
search_metadata	Request id, status, timestamps, and links back to the raw HTML/JSON of this search
search_parameters	Echoes back the parameters used: engine, q, sort_by, etc.
search_information	Result counts, response time, and store/location context (postal code, city, store id)
organic_results	The actual list of matched products — title, price, rating, images, variants, links
trending_items	Additional trending/related product suggestions
filters	Facets available for refining the search (category, brand, price range, etc.)
related_searches	Suggested related queries, each with a query string and a link
pagination	Info for paging through further result pages



Top-level JSON tree in the viewer, matching the table above.



Drilling into related_searches: each entry is a {query, link} pair pointing back to a new Walmart search URL.

Summary of the full flow

1. Pick the target endpoint and required parameters from the SearchApi.io docs (Walmart Search).
2. Copy the account's API key from the SearchApi dashboard.
3. Build and test the full request URL (endpoint + query params + api_key).
4. In Azure Data Factory, create a REST linked service pointed at the SearchApi base URL, plus a Blob Storage linked service for the destination.
5. Create a REST dataset (RestResource1) whose Relative URL holds the query string, and preview the live response to confirm it works.
6. Build a pipeline with a Copy Data activity; set its Source to RestResource1 and debug-run it.
7. Add a JSON dataset (Json1) on the Blob Storage linked service as the Sink, and debug-run again.
8. Confirm the resulting JSON blob landed correctly in the storage container.
9. Use a JSON viewer to explore the response structure (search_metadata, organic_results, related_searches, etc.) for downstream parsing or analysis.